BAT OCR Pipeline

How POD PDFs become invoice numbers, end to end.

This document describes the OCR worker as it exists in `v2026.4.13`. It covers the architecture, the per-row pipeline, the OCR engines and their ordering, the lane lifecycle, the defensive timeout layers, and how to read the System Log when something goes wrong.

---

1. Why this design

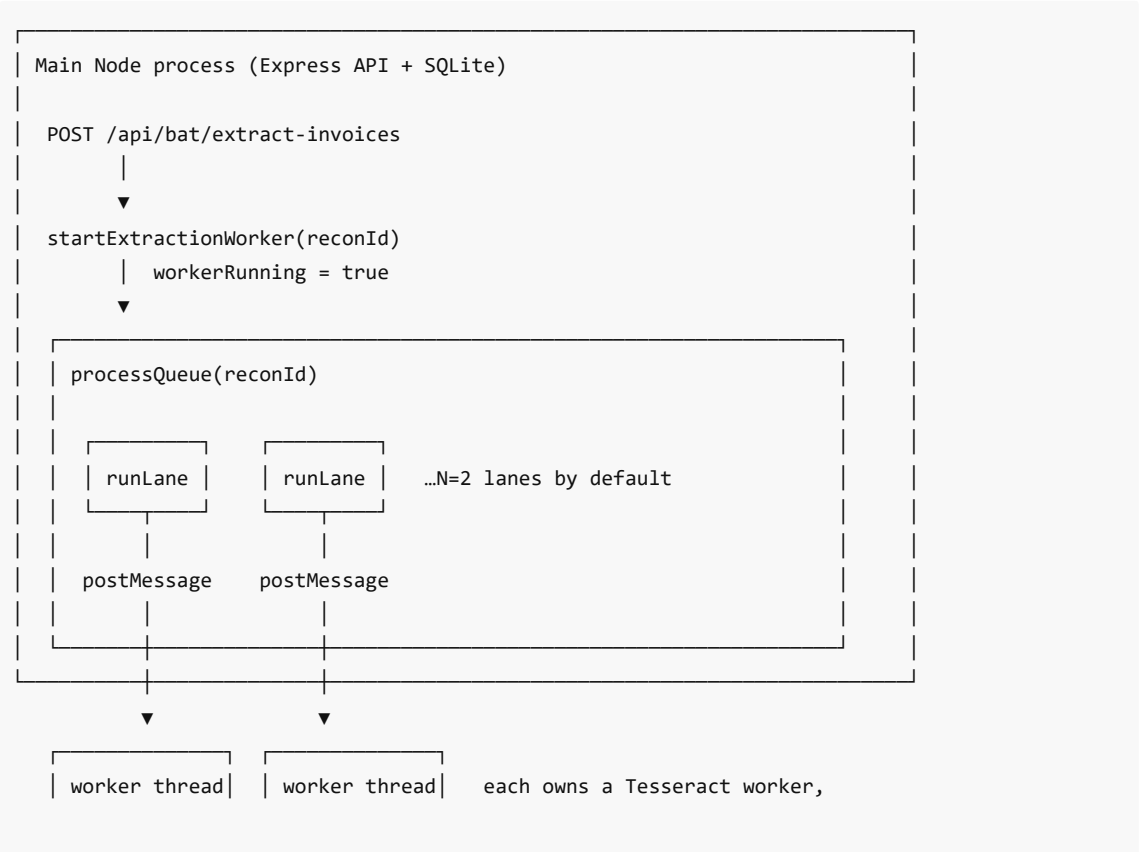
A reconciliation week typically has **150–300 POD PDFs**, each ~1–3 MB, served from SharePoint / Microsoft 365 URLs that the BAT supplier supplies. We need an invoice number out of each PDF. Three constraints shape the design:

1. **Mixed PDF quality.** Some PDFs have a real text layer (we can extract the invoice number directly without OCR). Most are scanned images. Some are skewed, rotated, or low-resolution.
2. **OCR is CPU- and memory-heavy.** A naive in-process implementation locks up the Node event loop while a single PDF renders or while Tesseract recognises an image. The whole API freezes for the duration of the run.
3. **No single OCR engine is reliable.** Google Vision is best but rate-limited and sometimes flat-out wrong. ocr.space has tier limits. Tesseract is slow and often misreads. The pipeline must try several engines in order.

The design that emerged: a **worker_thread pool** owned by the main Node process, with a **single PDF-per-lane discipline**, a **multi-engine cascading OCR pipeline**, and a **layered timeout + watchdog system** that guarantees no row can wedge the application.

---

2. High-level architecture



```
   │ (ocrWorker)  │  │ (ocrWorker)  │   sharp pipeline, pdfjs render
   └──────────────┘  └──────────────┘
```

Key boundaries:

- **Parent main thread** runs Express, SQLite (better-sqlite3 is synchronous), the SSE streams, and the orchestration logic in `processQueue`. It must stay responsive.
- **Worker threads** do the CPU- and IO-heavy work: PDF download, pdfjs render, sharp preprocessing, Tesseract recognition, OCR-engine API calls.
- **Communication is by `postMessage`** with a numeric `id` for correlation. No shared memory, no shared DB handle. The worker has zero database access.
- **One PDF per lane at a time.** A lane never has more than one in-flight extraction. This makes the timeout and watchdog logic tractable.

---

3. The lane: `OcrLane` class

A "lane" is one worker_thread plus the parent-side bookkeeping needed to dispatch tasks to it and observe its progress. The class lives in `src/services/batReconciliation.js`.

State per lane:

- `worker` — the actual Node `worker_thread` instance.
- `pending` — `Map<id, { resolve, reject, onProgress }>` for in-flight extractions. In practice never has more than one entry, but written as a Map for safety.
- `nextId` — monotonic counter for message correlation.
- `dead` — flag that flips to true when the worker has crashed, timed out, or been terminated.

Methods:

- `extract(payload, timeoutMs, onProgress)` — dispatch a single extraction. Returns a Promise that resolves with the result or rejects on timeout / worker crash.
- `terminate()` — clean shutdown: post a `shutdown` message to the worker, wait 500 ms for it to flush, then race Node's `worker.terminate()` against a 3-second hard cap (see §7.4 for why).

Message types (parent ↔ worker):

Direction	Type	Meaning
Parent → worker	`extract`	"Process this PDF, here's the payload, here's the id"
Parent → worker	`shutdown`	Clean exit (flush Tesseract, then `process.exit(0)`)
Worker → parent	`ready`	Posted once on worker startup
Worker → parent	`progress`	"Now in stage X" — fired multiple times per extraction
Worker → parent	`result`	Final outcome: `{ ok: true, result }` or `{ ok: false, error }`

---

4. The per-row pipeline

For a single row (one POD PDF), here's what happens inside the worker.

Stage 1: `download`

```
const response = await fetchWithTimeout(pdfUrl, {}, 60_000);
const buffer = Buffer.from(await response.arrayBuffer());
```

- Downloads the POD from its SharePoint / Microsoft 365 URL.
- 60-second timeout via `AbortController` (see §7.1). A stalled connection here used to wedge a whole lane until the parent's old 120 s backstop fired.
- If the response is not 200 or the body doesn't start with `%PDF`, the row is short-circuited as `not_found` with no error.

Stage 2: `pdf_text`

```
const pdfDoc = await pdfjsLib.getDocument({ data: new Uint8Array(buffer) }).promise;
for (let p = 1; p <= Math.min(pdfDoc.numPages, 3); p++) {
  const textContent = await page.getTextContent();
  // ...findInvoiceNumber...
}
```

- Tries to read the PDF's text layer (no OCR). Works for ~30 % of PODs that have an embedded text layer rather than just a scan.
- Up to the first 3 pages.
- If we find an invoice number here, we short-circuit and **skip the rest of the pipeline entirely**. This is the cheapest happy path — no rendering, no engine calls.
- If `pdfjs.getDocument` itself throws (corrupt PDF, encrypted, etc.), we silently fall through to image render.

Stage 3: `render`

```
imageBuffer = await pdfPageToImage(buffer, 1, isLarge ? 1.5 : 2.0);
```

- Renders page 1 to a PNG using **pdfjs-dist + node-canvas**.
- Render scale: 2.0× normally, 1.5× for PDFs > 2 MB (memory pressure).
- **`pdfjs-dist` is pinned at 4.8.69**. Newer versions use browser-only `ImageBitmap` APIs that node-canvas rejects with "Image or Canvas expected". See `docs/plans/pdf-engine-migration.md` for the long-term plan to migrate off node-canvas.

Stage 4: `preview_save`

```
const previewJpeg = await sharp(imageBuffer).resize({ width: 1200 }).jpeg({ quality: 70
}).toBuffer();
await fsp.writeFile(path.join(previewDir, `${extractionId}.jpg`), previewJpeg);
```

- Best-effort save of a JPEG preview at `${extractionId}.jpg` so the UI can show the OCR'd image alongside the row.
- Errors here are swallowed — preview is non-essential.

Stage 5: OCR engines (the cascade)

The engines are tried **in this order**, with both `0°` and `90°` rotations attempted before moving on:

#	Engine	Why this position

1	**GoogleVision**	Highest accuracy, fastest end-to-end. Skipped if `GOOGLE_VISION_KEY` not configured.
2	ocr.space engine 1	Cheap and decent on clear scans.
3	ocr.space engine 3	Different recognition algorithm — sometimes catches what engine 1 misses.
4	Tesseract (with threshold preprocessing)	Free, works offline, slow. Sharp pipeline: greyscale → normalise → sharpen σ=2 → threshold 140 → PNG.
5	Tesseract (no threshold)	Same as above without the threshold step — better for low-contrast scans.
6	ocr.space engine 2	Last resort — engine 2 has a tier-error rate but occasionally rescues edge cases.

Each engine attempt:

```
const stageLabel = `engine:${engine.name}@${angle}`;
emitProgress(msgId, stageLabel);
const text = await withTimeout(engine.run(angle), 60_000, stageLabel);
if (!text || text.trim().length < 20) continue;
const invoice = findInvoiceNumber(text);
if (invoice) return { invoice, previewPath };
```

- Stage label includes engine name and rotation so the operator can see exactly which combination is currently running (and which one wedged, if it does).
- 60-second per-engine timeout. Failure tells us *which* engine wedged.
- Text < 20 chars is treated as "engine failed silently" — fall through.
- `findInvoiceNumber(text)` is a regex pipeline (see §5).
- First match wins.

Stage 6: `done` or `failed`

- `done` — emitted just before the result message on success.
- `failed` — emitted on any throw before the result message.

If no engine produces a valid invoice number, the result is `{ invoice: null, previewPath, error: tierError }` — a `not_found` outcome with the most-recent tier error, if any.

---

5. Invoice-number extraction (`findInvoiceNumber`)

Cardoso invoice numbers have the form `IN<digits>`, where the digit count is one of `\{6, 7, 8, 9\}`. The regex pipeline normalises common OCR misreads then tries multiple patterns:

1. **Cleanup pass**: `|` → `I`, lowercase `o` / `O` between digits → `0`, `lI` → `I` before `N\d`, strip `*`, `{}[]()`, collapse `--` to space.
2. **Long form**: `\b(18\d{8,9})\b` → `IN<rest>`. Matches the 18000xxxxxx format that BAT actually uses.
3. **`IN`-prefixed long**: `\bIN\s*(\d{8,9})\b` — handles spaces between IN and digits.
4. **Partial-prefix patterns**: `\b\d?0{2,3}(422\d{3,6})\b` and similar — used when the leading digits are smudged.

5. `IN` **-prefixed short**: `\bIN\s*(\d{4,7})\b` , `\bINV\s*…` , `[IL1]\s*N\s*…` — covers OCR confusion of `I` with `L` / `1` and other artefacts.
6. **Standalone** `55xxxxx` — the legacy format.

If nothing matches, returns `null` and the caller falls through to the next engine.

---

6. Concurrency model

- `OCR_CONCURRENCY` **env var, default 2.** Each lane is a worker_thread that owns its own Tesseract worker, sharp pipeline, and pdfjs renderer.
- Why 2 by default: on a constrained Windows site, 4 parallel lanes saturate every CPU core and the main API thread starves for context-switch slots even though it isn't directly blocked. 2 keeps OCR throughput roughly the same (most time is in network calls to Google Vision / ocr.space) while leaving CPU headroom for the rest of the app.
- A queue of pending rows (`SELECT … WHERE extraction_status = 'pending'`) is drained by all lanes in parallel via the in-memory `inFlight` Set, which prevents two lanes from claiming the same row.

---

7. Defensive layers — why the pipeline can't silently hang

There are five independent safeguards. Each catches a different failure mode.

7.1 Per-fetch `AbortController` timeouts (worker-side)

Network call	Timeout
PDF download	60 s
Google Vision API	45 s
ocr.space API	60 s
Tesseract trained-data download (`createWorker('eng')`)	30 s

A stuck SharePoint connection, a blocked CDN, or a hung corporate proxy all surface as `Timeout in stage: <label>` instead of an indefinite wait.

7.2 Per-engine timeout

```
const text = await withTimeout(engine.run(angle), 60_000, stageLabel);
```

If a single engine attempt (download → preprocess → recognise) doesn't return in 60 s, we move on. This catches Tesseract's `recognize()` hanging on a pathological image, which `AbortController` can't help with.

7.3 Top-level extract deadline

```
const result = await withTimeout(
  extractInvoiceFromPdf(...),
  90_000,
  'extract_total',
);
```

Inside the worker's message handler. Even if every per-stage timeout fires sequentially you can't exceed ~90 s of real work for a single PDF on a healthy site. 90 s is the operator-visible "row time-out" — if a row is taking longer, something is broken in the environment, and that fact reaches the parent loud rather than hanging for minutes.

7.4 Parent-side watchdog (`HARD_KILL_THRESHOLD_MS`)

The slow-row monitor (a `setInterval` in the parent) scans `currentlyProcessing` every 30 seconds. Any row past **180 s** (i.e. `PDF_TIMEOUT + 60`) gets:

- A `bat.ocr.watchdog_kill` System Log entry that includes the wedged stage name.
- A force `lane.worker.terminate()` call.

This is the backstop for "the lane's internal `setTimeout` should have rejected but didn't" cases.

7.5 `lane.terminate()` 3-second race (the load-bearing fix from v2026.4.13)

```
await Promise.race([
  this.worker.terminate().catch(() => {}),
  new Promise((r) => setTimeout(r, 3000)),
]);
```

Discovered 2026-05-03 from the production System Log: when the worker thread is wedged in **native code** (Tesseract / sharp / canvas inside a syscall the OS can't preempt), Node's `worker.terminate()` returns a Promise that **never resolves**. The `await` used to hang forever, which meant `runLane` 's outer `finally` never ran and `currentlyProcessing.delete()` never fired. Symptom: rows climbing 120 s → 810 s → 1290 s with no new events, "Pause OCR" silently ignored, only a server restart unwedged it.

The 3-second race resolves the await regardless. The worker is dead either way once we've called `terminate()` ; we don't need to wait for OS-level cleanup. The race unblocks runLane → row leaves `currentlyProcessing` → the queue keeps moving → app stays responsive.

---

8. Diagnostics

8.1 System Log entries

In order of usefulness when something is wrong:

Source	Meaning
`bat.ocr.self_test`	Boot-time smoke test. Fires once on every server start. **Info** entry confirms OCR pipeline can spawn a worker thread and receive its `ready` message within 10 s. **Error** entry includes a hint: native module corrupt / AV-quarantined / Node version mismatch.
`bat.ocr.start`	Posted at the start of every `processQueue` run. Includes concurrency, platform, arch, Node version, whether each API key is set, `previewDir`. One-line snapshot of "is the environment right for OCR."
`bat-ocr.worker`	"Worker started for reconciliation X" / "Worker stopped (queue drained / paused mid-run / stopped with N pending) for reconciliation X".
`bat.ocr.row_slow`	Row has been in-flight > 60 s. Repeats every 60 s while the row is still stuck.

`bat.ocr.watchdog_kill`	Watchdog force-killed a lane after 180 s. Includes the wedged stage name and how long the row was in that stage.
`bat.ocr.lane_recreate`	Tried to recreate a dead lane and failed. Recon will be flagged `status='error'`.
`bat.ocr.run_incomplete`	All lanes retired before draining the queue. Recon flipped to `status='error'` with `last_error` populated for the UI.
`bat.ocr.row`	Per-row failure log. Includes pdf_url, error code, error class, lane state.
`bat.ocr.row_update`	Row UPDATE failed (extremely rare — better-sqlite3 contention).
`bat.ocr.memory`	Every 60 s during a run: RSS, heap, native, in-flight count, t+seconds. Suppressed when nothing has changed by 5 MB.
`bat.ocr.worker_thread`	Worker thread emitted `error` or exited with non-zero code.
`bat-ocr.pause` / `bat-ocr.resume`	Operator paused/resumed OCR.

8.2 Per-stage progress messages → in-flight UI

The worker emits `{ type: 'progress', id, stage }` as it transitions between stages. The parent's `OcrLane` forwards these to the slot's `onProgress` callback in `runLane`, which updates the row's entry in `currentlyProcessing`:

```
{
  id: 27,
  store_name: 'TOTAL BONJOUR …',
  reconciliation_id: 12,
  started_at_ms: 1714838418000,
  stage: 'engine:GoogleVision@0',
  stage_at_ms: 1714838434000,
  // ...
}
```

`getExtractionProgress(reconId)` exposes this in the `in_flight` array, and `ExtractionProgress.jsx` renders it inline in the live "Currently processing" panel. Stage badge turns red if a row sits on one stage for ≥ 60 s.

8.3 Reading a failure

When OCR misbehaves, this is the order to look at System Log entries:

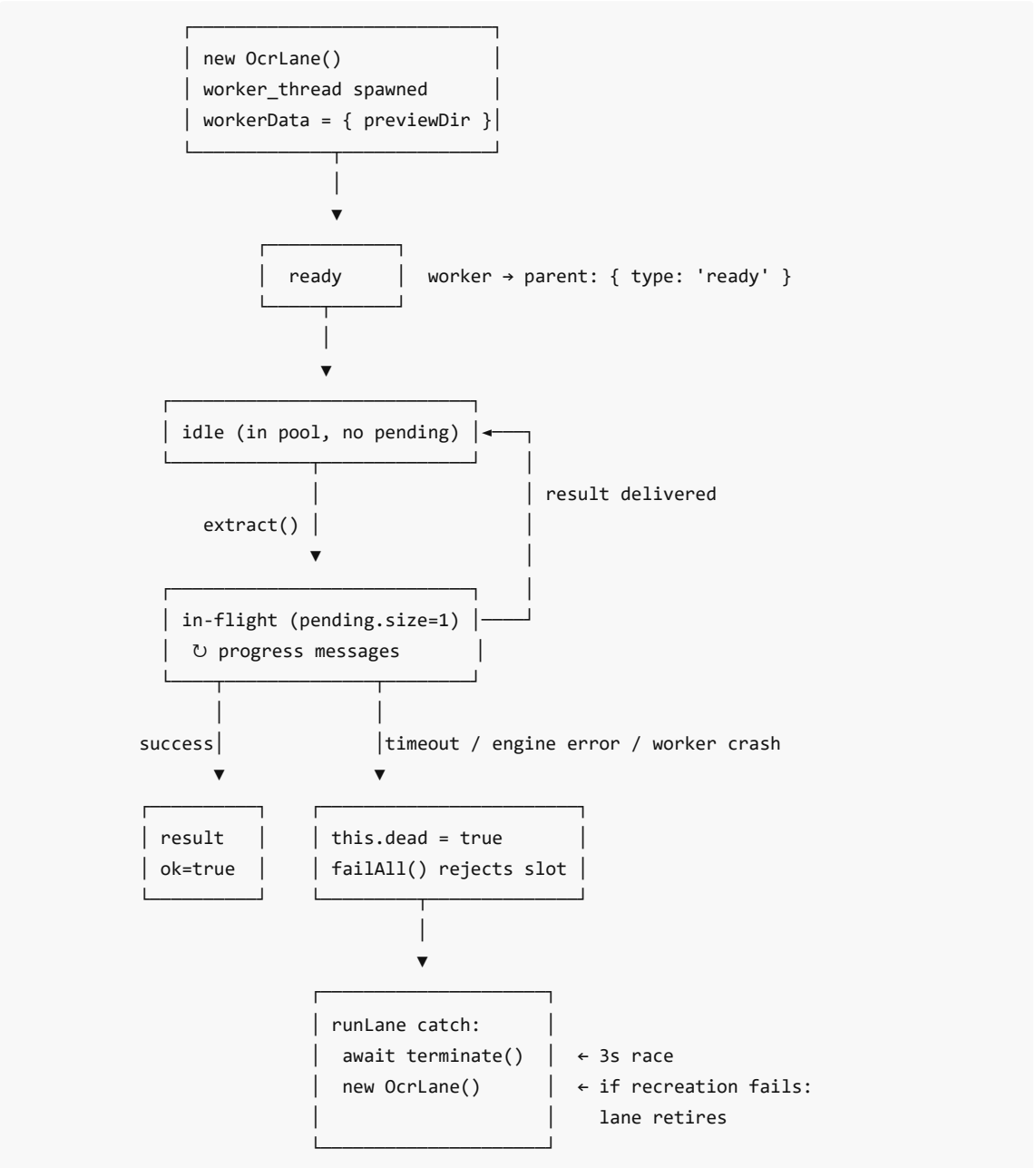
1. Was `bat.ocr.self_test` an **info** or **error** entry on the most recent boot?
   - Error → environment problem (native modules / AV / Node version). Fix the environment first.
   - Info → pipeline can spawn. Move on.
2. What does `bat.ocr.start` show for the current run?
   - `google_vision_key_set: false` and `ocr_space_key_set: false` → all PODs fall through to Tesseract, which then has to download trained data. If `tessdata.projectnaptha.com` is firewalled, see step 3.
3. Any `bat.ocr.watchdog_kill` entries?
   - Read the `stuck_stage` field. That's the engine that wedged.

- `engine:Tesseract@0` → almost always Tesseract trained-data download or native binary issue.
- `engine:GoogleVision@0` / `engine:ocr.space/*` → outbound HTTPS to that API is blocked or the API is rate-limiting.
- `download` → SharePoint / M365 PDF URL is unreachable from the site (auth issue, network).
- `render` → node-canvas issue (rare; usually a corrupted PDF).

4. Any `bat.ocr.run_incomplete`?
   - All lanes retired. Look up by `extraction_id` what the last `bat.ocr.row` errors were.

---

9. Lane lifecycle (the state machine)



---

10. The `runLane` work loop (parent-side)

Each lane runs this loop until the queue drains or the operator pauses:

```
while (true) {
  if (ocrPaused) return;
  const next = claimNext();           // returns null when pending=0
  if (!next) return;

  currentlyProcessing.set(next.id, {  // visible to the in-flight UI
    id, store_name, reconciliation_id,
    started_at_ms, lane, stage: 'queued', stage_at_ms,
  });

  try {
    try {
      const result = await lane.worker.extract(payload, PDF_TIMEOUT, onProgress);
      // success: write extracted_invoice, status='found' (or 'not_found')
    } catch (err) {
      // log + write status='failed', recordReconciliationError
      if (lane.worker.dead) {
        await lane.worker.terminate();   // 3s race
        lane.worker = new OcrLane();     // recreate or retire
      }
    }
  } finally {
    inFlight.delete(next.id);
    currentlyProcessing.delete(next.id);  // ← the line that didn't run pre-v4.13
    lastWarnedAt.delete(next.id);
  }
}
```

The outer `finally` is what frees the slot for the next row. **Anything that blocks between catch and finally blocks the entire queue.** v2026.4.13 fixes the one such blocking call (the unbounded `worker.terminate()`).

---

11. Pause / resume

- Persisted in `bat_settings` (key: `ocr_paused`, value `'0' | '1'`) so restarts don't silently reset the operator's choice.
- Default-paused on a fresh DB so a brand-new install doesn't burn OCR credits before keys are configured.
- While paused: no new workers start, auto-resume on server boot is skipped, and the in-flight `processQueue` loop **exits cleanly after its current invoice finishes** so we don't lose work mid-flight.
- Pause is checked at the top of `runLane`'s while loop. **Implication**: a wedged lane will not see the pause until it gets back to the loop top — which is why the v2026.4.13 `lane.terminate()` race is what actually makes pause responsive.

---

12. Auto-resume on boot

`resumeExtractionWorker()`, called from `server.js` at the end of boot:

1. Fires `runOcrSelfTest()` fire-and-forget regardless of pause state. Logs `bat.ocr.self_test`.
2. If OCR is paused, returns. No auto-resume.
3. Otherwise looks for any reconciliation with `extraction_status = 'pending'` rows. If found, calls `startExtractionWorker(reconId)` to pick up where the previous run left off.

This means a server restart mid-run is non-destructive: pending rows stay pending in the database, and on the next boot they get picked up automatically (or wait, if OCR is paused, until the operator resumes).

Appendix A: Files of interest

```
src/services/
├── batReconciliation.js     The OcrLane class, processQueue, runLane,
│                            slow-row + memory monitors, watchdog,
│                            startExtractionWorker, resumeExtractionWorker,
│                            runOcrSelfTest, getExtractionProgress.
├── ocrWorker.js             Runs in the worker_thread. extractInvoiceFromPdf,
│                            the OCR-engine wrappers, findInvoiceNumber,
│                            withTimeout / fetchWithTimeout / emitProgress.
src/components/reconciliation/
├── ExtractionProgress.jsx   Renders the live in-flight panel with stage
│                            attribution, slow-row colour rules.
└── InvoiceMatching.jsx      Per-row table on the recon detail page.
src/routes/
└── batReconciliation.js     /api/bat/extract-invoices,
                             /api/bat/extraction-status (poll),
                             /api/bat/extraction-status-stream (SSE).
```

Appendix B: Tunables

Constant	Value	Where defined	Notes
`OCR_CONCURRENCY`	2 (env-overridable)	`services/batReconciliation.js`	Number of lanes
`PDF_TIMEOUT`	120 000 ms	`services/batReconciliation.js`	Per-extraction in-extract timer
`HARD_KILL_THRESHOLD_MS`	`PDF_TIMEOUT + 60_000` (180 000 ms)	`services/batReconciliation.js`	Watchdog backstop
`SLOW_ROW_THRESHOLD_MS`	60 000 ms	`services/batReconciliation.js`	When `bat.ocr.row_slow` starts firing
Tesseract init timeout	30 000 ms	`services/ocrWorker.js`	`createWorker('eng')` cap
Top-level extract timeout	90 000 ms	`services/ocrWorker.js`	`extract_total` deadline
Per-engine timeout	60 000 ms	`services/ocrWorker.js`	Each `engine.run(angle)`

PDF download timeout	60 000 ms	`services/ocrWorker.js`	`fetchWithTimeout(pdfUrl, …, 60000)`
Google Vision timeout	45 000 ms	`services/ocrWorker.js`	API call cap
ocr.space timeout	60 000 ms	`services/ocrWorker.js`	API call cap
`lane.terminate()` race cap	3 000 ms	`services/batReconciliation.js`	The v2026.4.13 root-cause fix
Slow-row monitor interval	30 000 ms	`services/batReconciliation.js`	`setInterval` cadence
Per-row warn rate-limit	60 000 ms	`services/batReconciliation.js`	Each row warns at most once per 60 s
Memory monitor interval	60 000 ms	`services/batReconciliation.js`	`bat.ocr.memory` ticks

Appendix C: Glossary

- **Lane** — one worker_thread + parent-side bookkeeping. The unit of OCR concurrency.
- **POD** — Proof Of Delivery. The PDF the BAT supplier provides for each delivery, which contains a Cardoso invoice number we need to extract.
- **`currentlyProcessing`** — module-level `Map` of in-flight rows. The source of truth for "what is OCR doing right now" that the UI's in-flight panel reads.
- **`workerRunning`** — module-level boolean. True while `processQueue` is executing for any reconciliation. Prevents concurrent runs.
- **Stage** — a coarse label for what the worker is currently doing (`download` / `pdf_text` / `render` / `preview_save` / `engine:NAME@ANGLE` / `done` / `failed`). Emitted by the worker, attached to the row in `currentlyProcessing` , rendered in the UI.
- **Watchdog** — the slow-row monitor's hard-kill code path. Fires at 180 s into a row's runtime to terminate a lane the in-extract timer should have killed at 120 s.
- **Self-test** — the boot-time smoke test that verifies a worker_thread can spawn and reach its `ready` state. Logs `bat.ocr.self_test` .